RESUMEN DE ESTUDIO · PARCIAL

Programación 3

Concurrencia, Observer, MVC, clase Object, DAO/DTO y Serialización

Un resumen por tema con la misma estructura: concepto y propósito, diagrama UML y código Java mínimo. Cierra con un repaso rápido: glosario, cuadro comparativo y trampas clásicas.

Unidad	Temas
1	**Concurrencia**: Threads, Runnable, ciclo de vida, synchronized, monitores, wait/notify/notifyAll, bloqueo selectivo, semáforos
2	**Patrón Observer**: Observer–Observable, setChanged/notifyObservers, consecuencias
3	**Patrón MVC**: Modelo (dominio y aplicación), Vista, Controlador, MVC en Swing
4	**Clase Object**: Visión general, equals/hashCode, clone (superficial y profundo), compareTo
5	**DAO – DTO – JDBC**: Capa de acceso a datos, objeto de transferencia, los 5 pasos de JDBC
6	**Serialización**: Binaria (Serializable, transient) y XML (XMLEncoder/XMLDecoder)

NOTA Los 6 programas Java de este resumen fueron **compilados** antes de incluirlos y los 7 diagramas Mermaid fueron renderizados. Lo que no sale de tus apuntes está marcado como **extra**: el ejemplo de `Semaphore` (los apuntes solo lo nombran en el índice), el uso de `PreparedStatement` en vez de `Statement`, la excepción `InvalidClassException` y el contrato equals/hashCode explicado en detalle.

Índice

1. Concurrencia (Threads, synchronized, wait/notify) **3**
- 1.1 Concepto y Propósito **3**
- 1.2 Diagrama UML (Mermaid) **4**
- 1.3 Estructura en Código (Java) **5**

2. Patrón Observer (Observer – Observable) **7**
- 2.1 Concepto y Propósito **7**
- 2.2 Diagrama UML (Mermaid) **8**
- 2.3 Estructura en Código (Java) **8**

3. Patrón MVC (Modelo – Vista – Controlador) **9**
- 3.1 Concepto y Propósito **9**
- 3.2 Diagrama UML (Mermaid) **10**
- 3.3 Estructura en Código (Java) **10**

4. Clase Object: equals, clone y compareTo .....
- 4.1 Concepto y Propósito **12**
- 4.2 Diagrama UML (Mermaid) **13**
- 4.3 Estructura en Código (Java) **13**

5. DTO – DAO (y JDBC) **15**
- 5.1 Concepto y Propósito **15**
- 5.2 Diagrama UML (Mermaid) **16**
- 5.3 Estructura en Código (Java) **16**

6. Serialización (binaria y XML) **18**
- 6.1 Concepto y Propósito **18**
- 6.2 Diagrama UML (Mermaid) **19**
- 6.3 Estructura en Código (Java) **19**

Repaso Rápido **20**
- Glosario Rápido **20**
- Cuadro Comparativo Definitivo **20**
- Trampas Clásicas **21**

1. Concurrencia (Threads, synchronized, wait/notify)

1.1 Concepto y Propósito

Qué es. La programación concurrente expresa paralelismo entre tareas y resuelve los problemas de comunicación y sincronización entre ellas. En Java cada tarea corre en un **hilo** (`java.lang.Thread`). Al arrancar existe el hilo `main` y desde él se lanzan otros.

Problema que resuelve. Permite ejecutar partes del programa "a la vez", pero los hilos **comparten memoria**, y no se sabe en qué orden se ejecutan. Si dos hilos modifican el mismo objeto sin coordinarse, el resultado es impredecible (el ejemplo del contador devolvía 158049 en vez de 200000: la **sección crítica** `incrementa()` se interrumpía a mitad de ejecución).

Creación de hilos (dos formas):

Forma	Cómo	Ventaja / desventaja
Heredar de `Thread`	`class H extends Thread` + sobrescribir `run()`	Consume la única herencia (herencia simple)
Implementar `Runnable`	`class R implements Runnable` + `new Thread(new R()).start()`	Deja libre la herencia; es la preferida

- `start()` lanza el hilo y **retorna**; la JVM invoca `run()`. Solo se puede llamar **una vez** (si no: `IllegalThreadStateException`). Llamar `run()` directo NO crea un hilo nuevo.
- La implementación estándar de `Thread.run()` no hace nada.

Ciclo de vida: *New* → `start()` → *Runnable* → (`sleep()`, `wait()`, E/S) → *No runnable* → vuelve a *Runnable* → fin de `run()` → *Dead*. Otras causas de muerte: excepción no capturada, `stop()` y `destroy()` (estos dos, desaconsejados).

Monitores y `synchronized`.

- **Monitor**: objeto pensado para ser usado por varios hilos; sus métodos se ejecutan con **exclusión mutua** (a lo sumo un hilo dentro). Tiene: inicialización, métodos privados, métodos del monitor y **cola de entrada**.
- Cada objeto Java tiene un **bloqueo (lock)**. Un hilo que invoca un método `synchronized` adquiere el lock, ejecuta y lo libera al terminar (incluso si hay excepción). Otro hilo que invoque un método sincronizado del **mismo objeto** espera; los accesos **no** sincronizados no esperan.
- El lock es **por hilo** (reentrante): un método sincronizado puede llamar a otro sincronizado del mismo objeto sin bloquearse.
- **Métodos estáticos sincronizados**: bloquean la **clase**; solo excluyen a otros estáticos sincronizados, no a los métodos de instancia.
- Implicancias: los constructores **no** pueden ser `synchronized`; no hay garantía de orden entre hilos que compiten; una subclase puede redefinir un método sincronizado como no sincronizado.

Comunicación: `wait`, `notify`, `notifyAll` (definidos en `Object`, solo válidos dentro de código sincronizado). Sincronizar evita interferencia, pero no coordina: en Productor–Consumidor hay que hacer esperar al productor si la caja está llena y al consumidor si está vacía.

```
synchronized void m() throws InterruptedException {
    while (!condicion)   // 3) SIEMPRE while, nunca if
        wait();          // 2) suspende y libera el lock atómicamente
    ...                  // 1) todo dentro de código sincronizado
    notifyAll();         // 4) preferir notifyAll()
}
```

Qué hace `notifyAll()`: pasa a todos los hilos en `wait()` de *WAITING* a *BLOCKED* (cola de entrada del monitor). **No entrega el lock**: el emisor lo conserva hasta salir del bloque; luego los despertados compiten, y el ganador retoma **después del** `wait()` y reevalúa el `while`. `notify()` despierta uno cualquiera y solo es seguro si todos esperan la misma condición, a lo sumo uno se beneficia, y esto vale para todas las subclases.

Bloqueado vs. en `wait()`: bloqueado por `synchronized` = quiere entrar y otro tiene el lock. En `wait()` = ya estaba dentro, se suspendió voluntariamente y **soltó** el lock.

Sentencia `synchronized(obj) { ... }`. Región crítica más chica que un método, y puede bloquear sobre un objeto distinto de `this`. Un método `synchronized` equivale a envolver el cuerpo en `synchronized(this)`.

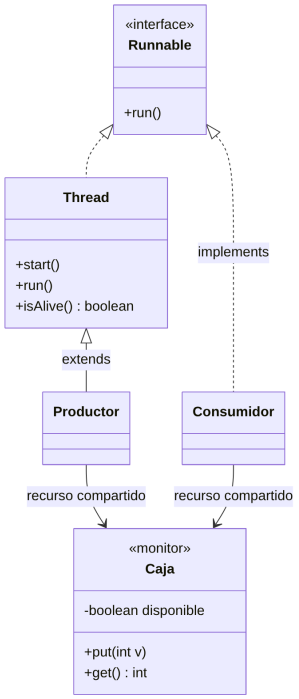
- *Ventajas*: granularidad fina y **bloqueo selectivo** (locks distintos para recursos independientes).
- *Desventaja*: si la usa el código **cliente** (el Thread), el objeto no puede "defenderse" solo; conviene usarla dentro del propio recurso.

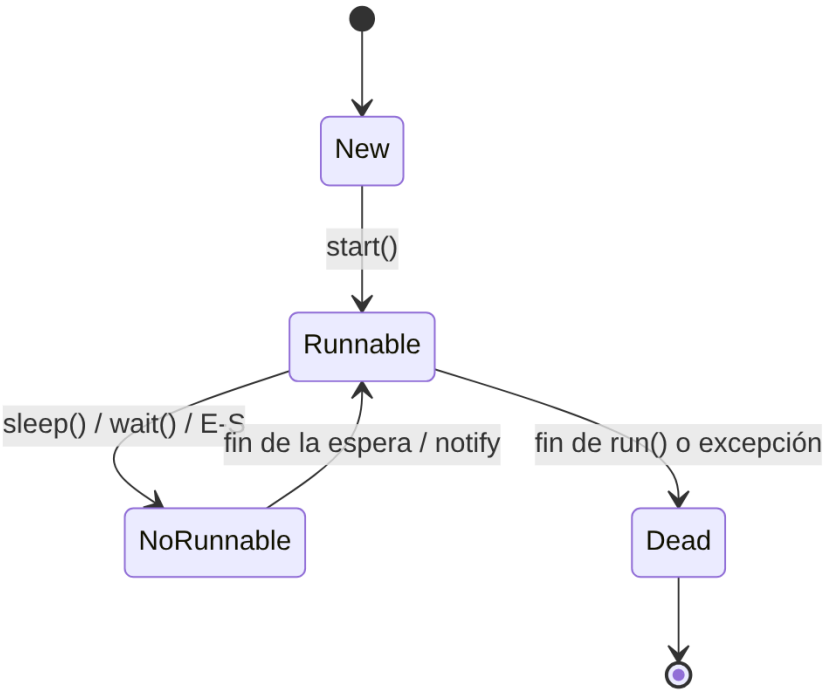
Semáforos (`acquire()` / `release()`): figuran en la agenda de la cátedra; se incluye un ejemplo mínimo de `java.util.concurrent.Semaphore`.

> **TIP DE PARCIAL** `synchronized` da **exclusión mutua**; `wait/notify` da **coordinación**. Si preguntan "¿por qué no alcanza con synchronized?": porque no resuelve la espera por una condición (caja llena / vacía).

> **OJO** `start()` se llama una sola vez y `run()` directo **no** crea un hilo nuevo. Los constructores no pueden ser `synchronized`.

1.2 Diagrama UML (Mermaid)





1.3 Estructura en Código (Java)

```java
import java.util.concurrent.Semaphore;

class Caja {                                    // MONITOR (recurso compartido)
    private boolean disponible = false;
    private int valor;

    public synchronized void put(int v) throws InterruptedException {
        while (disponible) wait();              // espera mientras esté llena
        valor = v;
        disponible = true;
        notifyAll();                            // avisa; el lock se libera al salir
    }

    public synchronized int get() throws InterruptedException {
        while (!disponible) wait();             // espera mientras esté vacía
        disponible = false;
        notifyAll();
        return valor;
    }
}

class Productor extends Thread {                // opción 1: herencia
    private final Caja caja;
    Productor(Caja c) { caja = c; }
    @Override public void run() {
        try { for (int i = 0; i < 5; i++) caja.put(i); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}

class Consumidor implements Runnable {          // opción 2: Runnable
    private final Caja caja;
    Consumidor(Caja c) { caja = c; }
    @Override public void run() {
        try { for (int i = 0; i < 5; i++) System.out.println(caja.get()); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}

class Contadores {                              // bloqueo selectivo
    private final Object lockA = new Object(), lockB = new Object();
    private int a, b;
    void incA() { synchronized (lockA) { a++; } }   // A y B no se bloquean entre sí
    void incB() { synchronized (lockB) { b++; } }
    static synchronized void estatico() { }          // lock de la CLASE Contadores
}
```

```java
class Recurso {                                  // semáforo binario
    private final Semaphore sem = new Semaphore(1);
    void usar() throws InterruptedException {
        sem.acquire();
        try { /* sección crítica */ } finally { sem.release(); }
    }
}

class Main {
    public static void main(String[] args) {
        Caja caja = new Caja();
        new Productor(caja).start();
        new Thread(new Consumidor(caja)).start();  // Runnable se envuelve en Thread
    }
}
```

2. Patrón Observer (Observer – Observable)

2.1 Concepto y Propósito

Qué es. Patrón **de comportamiento**. Define una dependencia 1-a-muchos entre objetos: cuando un objeto (**Observable/Sujeto**) cambia de estado, todos sus dependientes (**Observers**) son notificados y actualizados automáticamente. Nombres alternativos: *Dependants*, *Publish-Subscribe*.

Problema que resuelve. Mantener consistencia entre objetos relacionados **sin acoplarlos fuertemente** (ej.: separar la presentación de los datos de la aplicación). Se aplica cuando: una abstracción tiene dos aspectos dependientes entre sí; un cambio exige actualizar otros objetos sin saber cuántos son; o un objeto debe notificar a otros sin presuponer quiénes son.

Participantes:

- **Observable** (clase abstracta en Java): conoce a sus observadores (cantidad arbitraria) y ofrece añadir/quitar.
- **Observer** (interfaz): define `update(...)`, el método que se invoca al notificar.
- **Observable concreto**: guarda el estado de interés y notifica cuando cambia.
- **Observer concreto**: mantiene referencia al observado e implementa `update` para mantenerse consistente.

Implementación en Java (`java.util`).

- `Observable` es una **clase** (se hereda: `extends Observable`). `Observer` es una **interfaz** (`implements Observer`) con un solo método: `void update(Observable obj, Object param)`; `obj` es el observable que notifica, `param` describe el cambio.
- Métodos de `Observable`: `addObserver(o)`, `deleteObserver(o)`, `deleteObservers()`, `countObservers()`, `notifyObservers()` / `notifyObservers(param)`, y los de la bandera de cambio: `protected setChanged()`, `protected clearChanged()`, `hasChanged()`.
- **Flujo obligatorio:** cambiar estado → `setChanged()` → `notifyObservers(...)`. `notifyObservers` solo notifica **si la bandera está activa** y luego la limpia. Sin `setChanged()` no pasa nada.
- Es responsabilidad del **observador** vincularse al observado (`addObserver(this)`).

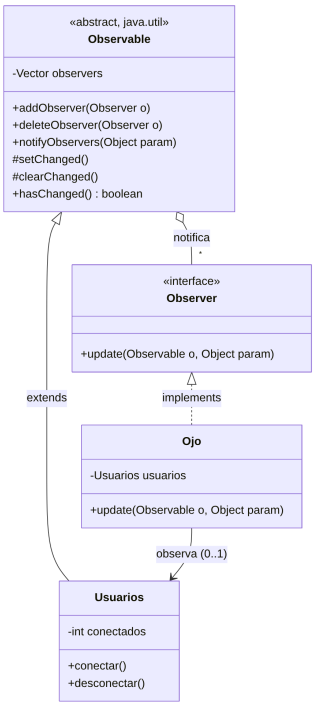
Consecuencias: observados y observadores varían y se reutilizan de forma independiente; se agregan observadores sin tocar el observado; acoplamiento abstracto/mínimo (el sujeto no conoce la clase concreta); soporte de *broadcast*; *actualizaciones inesperadas* (cascada de cambios, y el protocolo no dice **qué** cambió).

Decisiones de implementación: observar varios observados → pasar el sujeto en `update`; ¿quién llama `notify`? El sujeto (cómodo para el cliente, pero ineficiente si hay varios cambios seguidos) o el cliente (optimizable, pero más responsabilidad); evitar referencias colgantes a sujetos eliminados.

> Nota: `Observer/Observable` están *deprecated* desde Java 9, pero son los que usa la cátedra. Compilan (con advertencia).

> **OJO** Sin `setChanged()` antes de `notifyObservers()` **nadie es notificado**. Además `Observable` es una **clase**: heredarla consume tu única herencia.

2.2 Diagrama UML (Mermaid)



2.3 Estructura en Código (Java)

```java
import java.util.Observable;
import java.util.Observer;

class Usuarios extends Observable {            // OBSERVABLE concreto
    private int conectados;

    public void conectar() {
        conectados++;
        setChanged();                          // 1) marcar cambio (sin esto no notifica)
        notifyObservers("conectado");          // 2) invoca update() de cada observer
    }
    public void desconectar() {
        conectados--;
        setChanged();
        notifyObservers("desconectado");
    }
    public int getConectados() { return conectados; }
}

class Ojo implements Observer {                // OBSERVER concreto
    private final Usuarios usuarios;

    Ojo(Usuarios u) {
        usuarios = u;
        usuarios.addObserver(this);            // el observador se registra solo
    }

    @Override
    public void update(Observable o, Object param) {
        System.out.println(param + " -> total: " + ((Usuarios) o).getConectados());
    }
}

class Main {
    public static void main(String[] args) {
        Usuarios u = new Usuarios();
        new Ojo(u);
        new Ojo(u);                            // N observadores, el observable no los conoce
        u.conectar();
    }
}
```

3. Patrón MVC (Modelo – Vista – Controlador)

3.1 Concepto y Propósito

Qué es. Patrón **arquitectónico** (descrito por Trygve Reenskaug, 1979, Smalltalk) que separa los **datos y la lógica de negocio** de su **representación** y del módulo que **gestiona eventos y comunicación**. Base de Swing, J2EE, Struts, ASP.NET MVC, Rails, etc.

Problema que resuelve. Separa lógica de negocio de interfaz de usuario → ambas evolucionan por separado, se **reutilizan** más y se gana flexibilidad (varias vistas sobre el mismo modelo: reloj analógico y digital).

Componentes:

- **Modelo**: encapsula el estado de la aplicación, responde consultas, expone la funcionalidad y notifica a la vista de sus cambios. No sabe de vistas ni controlador. Se subdivide (Smalltalk/Swing) en:
- *Modelo del dominio*: clases del problema real (cliente, factura). No depende de nada externo.
- *Modelo de la aplicación* (coordinador): conoce las vistas y las **notifica** de los cambios del dominio.
- **Vista**: reproduce el modelo; toma de él solo lo que necesita mostrar y se actualiza cuando éste cambia. Envía los gestos del usuario al controlador.
- **Controlador**: dirige el flujo ante mensajes externos (clicks, menús); **modifica el modelo** y **selecciona/abre/cierra vistas**. Tiene acceso a modelo y vistas, pero éstos **no** conocen al controlador.

Flujo: (1) el usuario interactúa con la vista → (2) el controlador recibe el evento (event handler/listener) → (3) el controlador actualiza el modelo → (4) la vista se actualiza obteniendo datos del modelo (el modelo no la conoce directamente: se usa **Observer** como indirección) → (5) la interfaz espera la siguiente interacción.

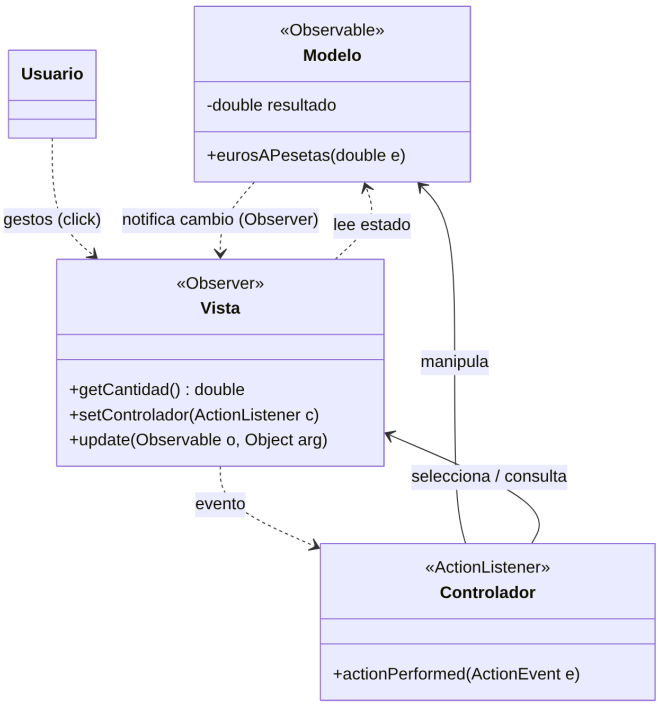
Relación con otros patrones. Vista–Modelo = **Observer**; controladores complejos usan **Command**. En aplicaciones **web** no se puede usar Observer (vistas desconectadas del modelo). Variante Apple/Cocoa: la vista no accede al modelo, el controlador le envía los datos.

MVC en Swing. Swing implementa MVC con diferencias: controlador y vista se fusionan en el **delegado de interfaz de usuario (UI delegate)**; cada componente se asocia a datos mediante un *modelo de aplicación* (ej. `AbstractTableModel`, que hace de **puente** entre el dominio — `Persona`, `ListaDePersonas` — y el `JTable`). Eventos: **modelo de delegación**: *fuentes* (botones, listas) disparan eventos y *listeners* los atienden; la fuente registra/elimina listeners. `setActionCommand()` identifica qué botón se pulsó y `getActionCommand()` lo lee en `actionPerformed`.

> **TIP DE PARCIAL** El **modelo no conoce** a la vista ni al controlador; la vista se entera por **Observer**. El controlador sí conoce a ambos.

> **OJO** En aplicaciones web no se puede usar Observer entre modelo y vista (están desconectados).

3.2 Diagrama UML (Mermaid)



3.3 Estructura en Código (Java)

```
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Observable;
import java.util.Observer;
import javax.swing.*;

class Modelo extends Observable {                // no conoce a Vista ni a Controlador
    public void eurosAPesetas(double euros) {
        double pesetas = euros * 166.386;
        setChanged();
        notifyObservers(pesetas);                // Observer: indirección hacia la vista
    }
}

class Vista extends JFrame implements Observer {  // solo muestra y reenvía gestos
    private final JTextField campo = new JTextField(8);
    private final JButton boton = new JButton("Convertir");
    private final JLabel salida = new JLabel("-");

    Vista() {
        setLayout(new FlowLayout());
        boton.setActionCommand("EUROS");         // identifica el botón
        add(campo); add(boton); add(salida);
    }
    double getCantidad() { return Double.parseDouble(campo.getText()); }
    void setControlador(ActionListener c) { boton.addActionListener(c); }

    @Override public void update(Observable o, Object arg) {   // el modelo avisa
        salida.setText(arg + " pesetas");
    }
}

class Controlador implements ActionListener {     // vincula acciones con el modelo
    private final Modelo modelo;
    private final Vista vista;

    Controlador(Modelo m, Vista v) {
        modelo = m; vista = v;
        modelo.addObserver(vista);               // vista observa al modelo
        vista.setControlador(this);              // controlador escucha a la vista
    }
    @Override public void actionPerformed(ActionEvent e) {
        if ("EUROS".equals(e.getActionCommand()))
```

```
            modelo.eurosAPesetas(vista.getCantidad());   // el controlador modifica el modelo
    }
}

class Main {
    public static void main(String[] args) {
        Modelo m = new Modelo();
        Vista v = new Vista();
        new Controlador(m, v);                    // ensamblado (inyección por constructor)
        v.pack();
        v.setVisible(true);
    }
}
```

4. Clase `Object`: `equals`, `clone` y `compareTo`

4.1 Concepto y Propósito

La clase `Object` en general. Es la **raíz de la jerarquía**: toda clase extiende directa o indirectamente de `Object`, por lo que una variable `Object` puede referir cualquier objeto (instancia o array). Define métodos heredados por todos: `equals`, `hashCode`, `clone` (protected), `getClass` (final), `finalize` (protected), `toString`.

`equals(Object obj)` (y su contrato con `hashCode`)

- Compara por **igualdad de valor**; `==` / `!=` compara **identidad** (misma referencia).
- La implementación por defecto es `this == obj`: un objeto solo es igual a sí mismo. Se **redefine** para dar una noción propia de igualdad (ej. `String` compara contenido).
- **Contrato:** si `a.equals(b)` es `true`, entonces `a.hashCode() == b.hashCode()`. Al redefinir `equals`, hay que redefinir `hashCode` (el hash es un entero de 32 bits derivado del estado/campos). Si no, las colecciones basadas en hash (`HashSet`, `HashMap`) fallan.
- Todo `equals` debe tomar `Object` como parámetro (con otro tipo es una sobrecarga, no una redefinición).

`clone()`

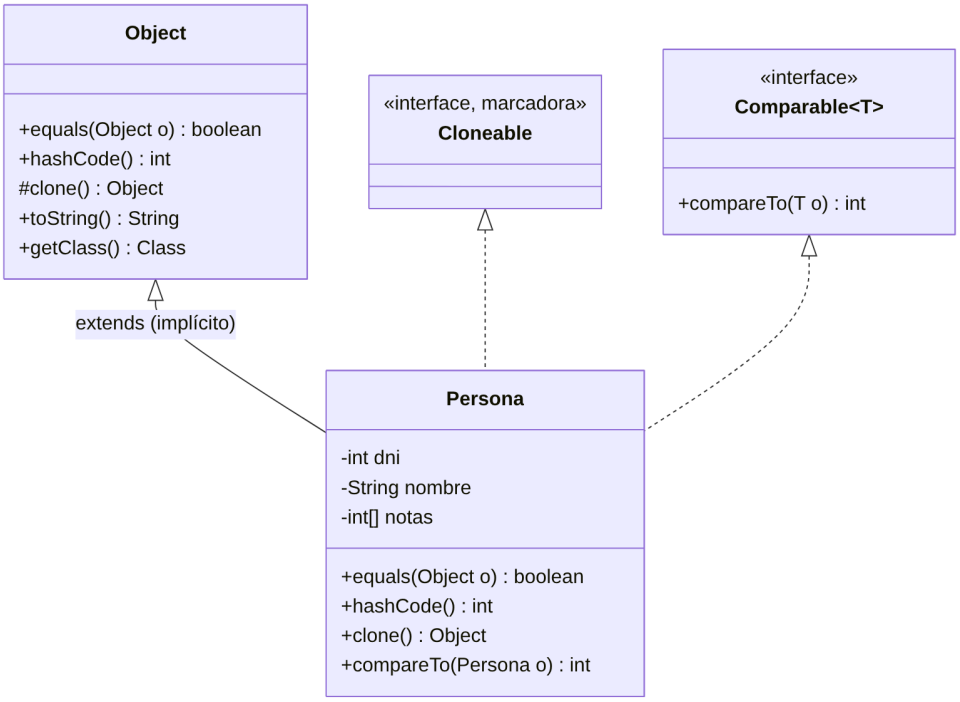
- **Clonar** = duplicar un objeto: dos objetos con el mismo estado inicial pero **identidad propia**; cambios posteriores en uno no afectan al otro.
- `Object.clone()` es `native` y `protected`: reserva memoria equivalente y copia campo a campo (copia **superficial**: las referencias se copian, no los objetos apuntados). Devuelve `Object` (requiere cast).
- Para usarlo: la clase implementa la interfaz **marcadora** `Cloneable` y redefine `clone()` como `public`. Si no es `Cloneable`, `super.clone()` lanza `CloneNotSupportedException` (excepción chequeada).
- Problema típico: una `PilaEnteros` clonada comparte el mismo `int[] buffer` con el original (los arrays son objetos). **Solución:** `super.clone()` y luego clonar manualmente cada atributo mutable (`nObj.buffer = buffer.clone()`) → copia **profunda**.
- **Nunca usar `new` dentro de `clone()`**: sería incorrecto para las subclases; por eso se empieza con `super.clone()`, que crea un objeto del tipo real.
- Solo hace falta código especial para los campos donde copiar el valor es incorrecto. Los atributos objeto deben soportar `clone()` o tener **constructor de copia** (`String` no tiene `clone()` pero sí constructor de copia).
- **Tres actitudes frente a `clone`:**
- **Admitir**: implementa `Cloneable` y declara `clone()` sin `throws`.
- **Admitir condicionalmente**: implementa `Cloneable` pero deja pasar la `CloneNotSupportedException` de sus componentes.
- **Prohibir**: no implementa `Cloneable` y `clone()` lanza siempre `CloneNotSupportedException`.
- Una subclase **no puede agregar** excepciones chequeadas a un método redefinido; por eso una clase que garantiza clonación elimina el `throws`.

`compareTo` (interfaz `Comparable<T>`)

- `Comparable<T>` tiene un único método: `int compareTo(T o)`. Define el **orden natural** de los objetos y permite **programación genérica**: un algoritmo (ej. inserción directa) restringe el tipo a `T extends Comparable<T>` y sirve para cualquier clase comparable.
- Convención: negativo si `this < o`, `0` si son iguales, positivo si `this > o`. Conviene que sea **consistente con `equals`** (`compareTo == 0` ⇔ `equals`).

TRAMPA CLÁSICA Dentro de `clone()` nunca uses `new`: empezá por `super.clone()`. Y `equals` recibe `Object`; con otro tipo es sobrecarga.

4.2 Diagrama UML (Mermaid)



4.3 Estructura en Código (Java)

```java
import java.util.Objects;

class Persona implements Cloneable, Comparable<Persona> {
    private final int dni;
    private final String nombre;
    private int[] notas = new int[3];                  // atributo mutable -> clon profundo

    Persona(int dni, String nombre) { this.dni = dni; this.nombre = nombre; }

    @Override                                          // parámetro Object, no Persona
    public boolean equals(Object o) {
        if (this == o) return true;                    // misma referencia
        if (!(o instanceof Persona)) return false;     // tipo incorrecto / null
        Persona p = (Persona) o;
        return dni == p.dni && Objects.equals(nombre, p.nombre);
    }

    @Override
    public int hashCode() {                            // coherente con equals
        return Objects.hash(dni, nombre);
    }

    @Override
    public Object clone() {                            // protected -> public
        try {
            Persona c = (Persona) super.clone();       // copia superficial, tipo real
            c.notas = notas.clone();                   // copia profunda del array
            return c;
        } catch (CloneNotSupportedException e) {       // no ocurre: implementa Cloneable
            throw new InternalError(e.toString());
        }
    }

    @Override
    public int compareTo(Persona o) {                  // orden natural por dni
        return Integer.compare(dni, o.dni);
    }
}
```

```
// "Prohibir clone": no implementar Cloneable y redefinir así:
// public Object clone() throws CloneNotSupportedException {
//     throw new CloneNotSupportedException();
// }
```

5. DTO – DAO (y JDBC)

5.1 Concepto y Propósito

DAO — Data Access Object

Qué es. Componente que ofrece una **interfaz común** entre la aplicación y uno o más dispositivos de almacenamiento (base de datos, archivo, etc.). Actúa como **adaptador/traductor** entre la lógica de negocio y la capa de persistencia: encapsula el código de acceso a datos de una fuente concreta.

Problema que resuelve. Desacopla la lógica de negocio de la tecnología de persistencia. Hay **un DAO por fuente de datos** (SQL Server, MySQL, archivos…); si la fuente cambia se cambia de DAO y el negocio no se entera. El DAO expone insertar, actualizar, borrar y consultar.

Componentes:

- **BusinessObject**: contiene la lógica de negocio; usa el DAO.
- **DataAccessObject**: capa de acceso a datos; oculta la fuente y los detalles técnicos.
- **TransferObject**: objeto plano (patrón **DTO**) que viaja entre el DAO y el negocio.
- **DataSource**: la fuente abstracta (BD, web service, LDAP, archivo de texto…).

Secuencia: (1) el BusinessObject obtiene el DAO → (2) le pide información → el DAO consulta el DataSource, **crea el TransferObject** y lo retorna → (3) el negocio modifica el TransferObject → (4) pide al DAO que guarde → el DAO persiste en el DataSource.

> **Error común:** no usar TransferObject y devolver los objetos propios de la API de la fuente (ej. `ResultSet`). Eso acopla el negocio a esas librerías: si cambia la fuente, cambia el negocio.

DTO — Data Transfer Object

Qué es. Objeto plano (**POJO**) con atributos que viajan en **una sola invocación** entre capas/cliente-servidor. Puede reunir datos de **varias tablas/fuentes** (ej. `fullName` = `firstName` + `lastName`) y **omitir** datos innecesarios.

Problema que resuelve. Evita usar las clases de entidad para devolver datos (que trae de más o obliga a múltiples viajes a la capa de servicios).

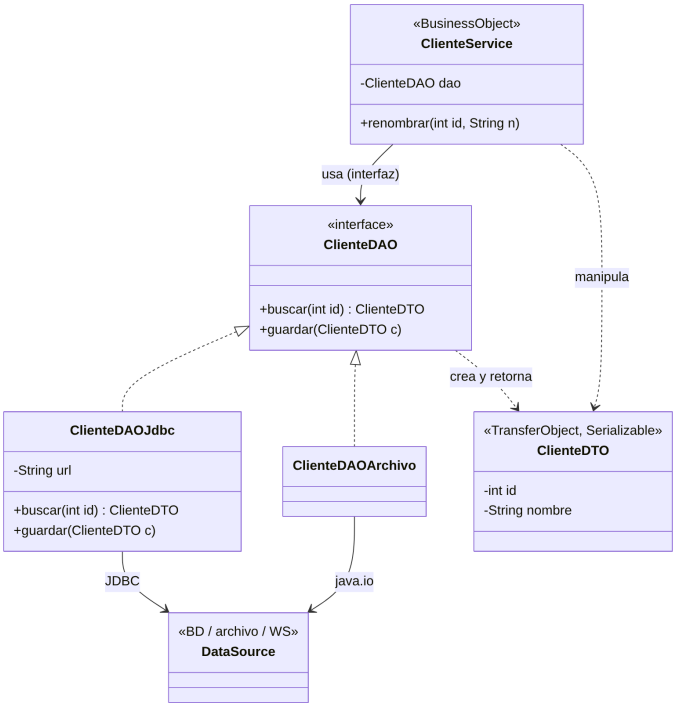
Reglas: solo datos (get/set, sin lógica de negocio) y **Serializable** (la clase y todos sus atributos), porque viaja por la red.

JDBC — cómo se implementa un DAO relacional

- **JDBC** (Java Database Connectivity, paquete `java.sql`, parte de Java SE) permite a Java conectarse a **bases relacionales** (MySQL, PostgreSQL, Oracle, H2…). Es independiente del gestor de BD, pero **no** independiente de SQL ni sirve para BD no relacionales.
- La aplicación habla solo con la **interfaz JDBC**; ésta usa el **driver** del gestor. Se necesita el driver del SGBD.
- **Los 5 pasos:** (1) cargar el driver (`Class.forName(...)`; en JDBC modernos el `DriverManager` lo detecta solo), (2) hacer la conexión (`DriverManager.getConnection(url, user, pass)`), (3) crear la sentencia (`Statement`; mejor `PreparedStatement`), (4) ejecutar (`execute` para DDL/insert, `executeQuery` → `ResultSet`; `while (rs.next())` itera), (5) cerrar la conexión (garantiza que se actualice la BD).
- Todo error JDBC se informa con `SQLException` (chequeada).

> **OJO** Error común del patrón: no usar TransferObject y devolver el `ResultSet` u otras clases de la API de la fuente. Acopla el negocio a esa librería.

5.2 Diagrama UML (Mermaid)



5.3 Estructura en Código (Java)

```
import java.io.Serializable;
import java.sql.*;

class ClienteDTO implements Serializable {          // TransferObject: POJO serializable
    private int id;
    private String nombre;
    ClienteDTO(int id, String nombre) { this.id = id; this.nombre = nombre; }
    int getId() { return id; }
    String getNombre() { return nombre; }
    void setNombre(String n) { nombre = n; }
}

interface ClienteDAO {                              // contrato: oculta la fuente
    ClienteDTO buscar(int id) throws Exception;
    void guardar(ClienteDTO c) throws Exception;
}

class ClienteDAOJdbc implements ClienteDAO {        // DAO para una fuente concreta (SQL)
    private final String url;                       // ej: "jdbc:h2:mem:test"
    ClienteDAOJdbc(String url) { this.url = url; }

    @Override
    public ClienteDTO buscar(int id) throws SQLException {
        try (Connection con = DriverManager.getConnection(url);             // paso 2
             PreparedStatement ps =
                 con.prepareStatement("SELECT id, nombre FROM CLIENTE WHERE id = ?")) { // paso 3
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {                        // paso 4
                return rs.next()
                    ? new ClienteDTO(rs.getInt("id"), rs.getString("nombre")) // crea el DTO
                    : null;
            }
        }                                                                   // paso 5: cierra todo
    }

    @Override
    public void guardar(ClienteDTO c) throws SQLException {
        try (Connection con = DriverManager.getConnection(url);
             PreparedStatement ps =
                 con.prepareStatement("INSERT INTO CLIENTE (id, nombre) VALUES (?, ?)")) {
            ps.setInt(1, c.getId());
            ps.setString(2, c.getNombre());
            ps.executeUpdate();
        }
```

```
    }
}

class ClienteService {                              // BusinessObject: solo conoce DAO + DTO
    private final ClienteDAO dao;                   // dependencia hacia la interfaz
    ClienteService(ClienteDAO dao) { this.dao = dao; }   // inyección: cambia la fuente sin tocar esto

    void renombrar(int id, String nuevo) throws Exception {
        ClienteDTO c = dao.buscar(id);              // 2) pide al DAO
        c.setNombre(nuevo);                         // 3) modifica el DTO
        dao.guardar(c);                             // 4) pide guardar
    }
}
```

6. Serialización (binaria y XML)

6.1 Concepto y Propósito

Qué es. Proceso de **codificar un objeto** (junto con todo su grafo de objetos referenciados) como una secuencia de bytes —o en un formato legible como XML/JSON— para **guardarlo** (archivo, BD, memoria) o **transmitirlo** por red. La **deserialización** reconstruye un objeto idéntico al original (un clon, con su estado interno).

Para qué sirve / problema que resuelve: persistencia (guardar el estado de un componente y restaurarlo luego) e **invocación remota de objetos** (transportar argumentos y valores de retorno). Los objetos referencian a otros: todos deben guardarse para mantener las relaciones.

Serialización binaria (nativa de Java)

- Está **orientada a bytes**: usa la jerarquía `InputStream`/`OutputStream`. Se envuelve un stream de destino en `ObjectOutputStream` y se llama `writeObject(obj)`; para leer, `ObjectInputStream.readObject()` (requiere cast).
- La clase debe implementar la interfaz **marcadora** `java.io.Serializable` (no hay métodos que implementar). **Todas** las variables de instancia deben ser serializables (primitivos y arrays lo son); si no → `NotSerializableException`.
- Se escribe: la clase, su firma y los valores de los miembros **no** `static` **ni** `transient`. Los `transient` se restauran con el valor por defecto (0, `null`, `false`). Los métodos `defaultWriteObject`/`defaultReadObject` hacen el trabajo por defecto.
- Al reconstruir, la clase debe estar en el **classpath**. Al deserializar **no se ejecuta el constructor** de la clase serializable.
- **Herencia:** basta con que la **clase base** implemente `Serializable` para que toda la jerarquía lo sea. Si la **superclase no es serializable**, sus atributos no se guardan y al deserializar toman valores por defecto (los que fije su constructor sin argumentos, que **debe existir** o falla con `InvalidClassException`).
- El archivo resultante es un formato **propio de Java**, no legible con un editor de texto. Es **frágil**: depende de la forma interna de la clase, y no admite cambios entre codificar y decodificar.

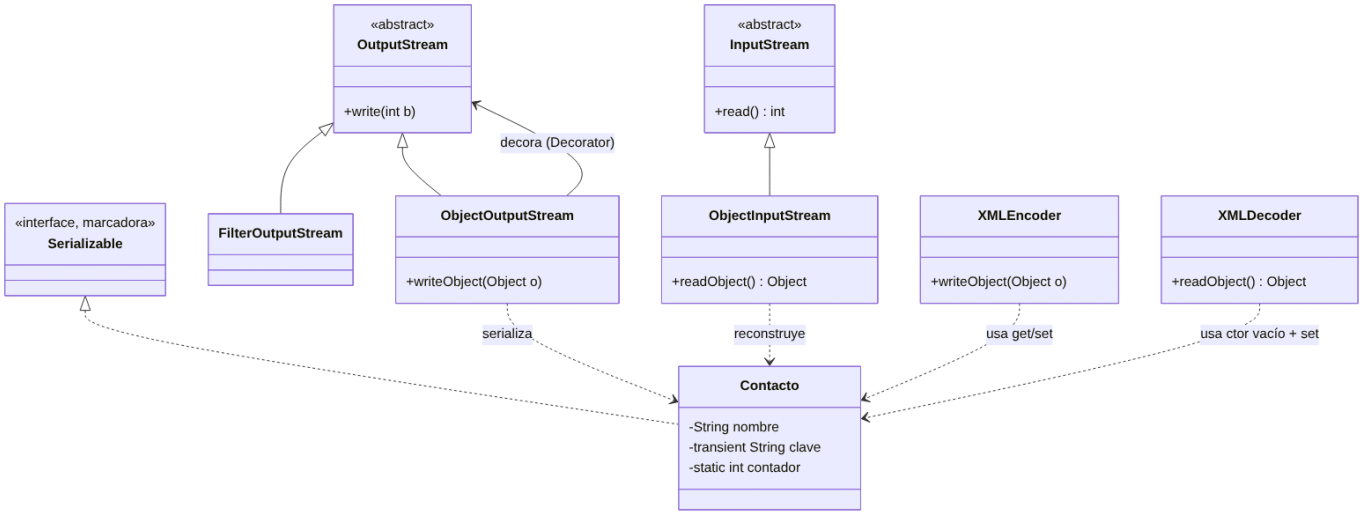
Serialización XML

- **XML** (eXtensible Markup Language, W3C): formato de marcas extensible y legible; estándar de intercambio **entre plataformas**; el analizador es estándar. Desde J2SE 1.4 los objetos se pueden serializar como XML.
- `java.beans.XMLEncoder` / `XMLDecoder`: en vez de volcar los bits internos, **registran los pasos necesarios para recrear el objeto vía su API pública**. Por eso es independiente de la versión de Java y menos frágil.
- **Requisitos de la clase (JavaBean): constructor sin argumentos**, estado expuesto con **get/set**, y propiedades **independientes** (sin efectos colaterales entre get/set de distintas propiedades). (Además debe ser `public`).

Patrón relacionado: los *filter streams* (`FilterInputStream`, `BufferedInputStream`…) agregan funcionalidad a un flujo existente: es el patrón estructural **Decorator**.

> **TRAMPA CLÁSICA** `transient` y `static` no se serializan. Un solo atributo no serializable produce `NotSerializableException`. Para XML: constructor vacío + get/set.

6.2 Diagrama UML (Mermaid)



6.3 Estructura en Código (Java)

> Guardar como `Contacto.java` (la clase pública debe llamarse igual que el archivo).

```
import java.beans.XMLDecoder;
import java.beans.XMLEncoder;
import java.io.*;

public class Contacto implements Serializable {      // marca: sin métodos que implementar
    private String nombre;
    private transient String clave;                  // NO se serializa -> vuelve null
    private static int contador;                     // static: NO se serializa

    public Contacto() { }                            // requerido por XMLEncoder/Decoder
    public String getNombre() { return nombre; }     // propiedades JavaBean
    public void setNombre(String n) { nombre = n; }
    public void setClave(String c) { clave = c; }
}

class Base { int x = 5; Base() { } }                 // NO serializable: necesita ctor sin args
class Derivada extends Base implements Serializable { int y; }   // x NO se guarda; se re-inicia

class Main {
    public static void main(String[] args) throws Exception {
        Contacto c = new Contacto();
        c.setNombre("Ana");
        c.setClave("1234");

        // Binaria: OutputStream -> ObjectOutputStream -> writeObject
        try (ObjectOutputStream out =
                 new ObjectOutputStream(new FileOutputStream("clientes.ser"))) {
            out.writeObject(c);
        }
        try (ObjectInputStream in =
                 new ObjectInputStream(new FileInputStream("clientes.ser"))) {
            Contacto r = (Contacto) in.readObject(); // cast; no se invoca el constructor
            System.out.println(r.getNombre());       // "Ana" (clave quedó en null)
        }

        // XML: guarda los pasos para recrear el objeto con su API pública
        try (XMLEncoder enc = new XMLEncoder(
                 new BufferedOutputStream(new FileOutputStream("contacto.xml")))) {
            enc.writeObject(c);
        }
        try (XMLDecoder dec = new XMLDecoder(
                 new BufferedInputStream(new FileInputStream("contacto.xml")))) {
            Contacto r = (Contacto) dec.readObject();
            System.out.println(r.getNombre());
        }
    }
}
```

Repaso Rápido

Glosario Rápido

- **Sección crítica:** segmento de código que accede a memoria compartida desde varios hilos y que debe ejecutarse de a un hilo por vez.
- **Monitor:** objeto cuyos métodos se ejecutan con exclusión mutua (un solo hilo dentro) y que dispone de cola de entrada y variables de condición (`wait`/`notify`).
- **`notifyAll()`:** despierta a todos los hilos en `wait()` sobre el objeto, pero no les cede el lock: lo recuperan compitiendo cuando el emisor sale del bloque sincronizado.
- **Observer – Observable:** patrón de comportamiento donde un sujeto notifica automáticamente (`setChanged()` + `notifyObservers()` → `update()`) a sus dependientes cuando cambia su estado.
- **MVC:** patrón arquitectónico que separa Modelo (estado y lógica), Vista (presentación) y Controlador (flujo y eventos), con el modelo ignorando a vista y controlador.
- **Clonación (`clone()`):** duplicación de un objeto en otro con identidad propia; `Object.clone()` es superficial, y los atributos mutables deben clonarse a mano para lograr una copia profunda.
- **Contrato `equals`/`hashCode`:** si dos objetos son `equals`, deben devolver el mismo `hashCode`, por lo que ambos métodos se redefinen siempre juntos.
- **DAO (Data Access Object):** capa-adaptador que encapsula el acceso a una fuente de datos concreta detrás de una interfaz común, aislando a la lógica de negocio.
- **DTO (Data Transfer Object):** POJO serializable, sin lógica, que agrupa datos (de una o varias fuentes) para transportarlos entre capas en una sola invocación.
- **Serialización:** codificación de un objeto y su grafo a bytes (o XML/JSON) para persistirlo o transmitirlo, excluyendo campos `static` y `transient`.

Cuadro Comparativo Definitivo

Concepto A	Concepto B	Diferencia clave para el examen
Extender `Thread`	**Implementar `Runnable`**	`Thread` consume la herencia simple; `Runnable` la deja libre y se pasa a `new Thread(r)`. En ambos, el hilo arranca con `start()`, no con `run()`.
Método `synchronized`	**Sentencia `synchronized(obj){}`**	El método bloquea `this` durante todo el método; la sentencia bloquea la región elegida y sobre cualquier objeto (bloqueo selectivo), pero delega la responsabilidad en el cliente.
Bloqueado por `synchronized`	**Suspendido por `wait()`**	Bloqueado: quiere entrar y no tiene el lock. `wait()`: ya estaba dentro, se suspendió y **liberó** el lock.
`notify()`	**`notifyAll()`**	`notify` despierta a uno cualquiera (riesgo de despertar al equivocado); `notifyAll` despierta a todos, que reevalúan su condición. Se prefiere `notifyAll`.
`while (!cond) wait()`	**`if (!cond) wait()`**	Con `if` se asume que despertar implica condición cierta (falso); con `while` se reevalúa. Siempre `while`.
Observable	**Observer**	`Observable` es una **clase** que se hereda (consume la herencia); `Observer` es una **interfaz** con `update(Observable, Object)`.
Observer	**MVC**	Observer es un patrón de comportamiento (relación 1-a-N entre objetos); MVC es arquitectónico (3 componentes) y usa Observer para el vínculo Modelo→Vista.

Concepto A	Concepto B	Diferencia clave para el examen
Modelo del dominio	**Modelo de la aplicación**	Dominio: clases del problema, sin conocer nada externo. Aplicación: coordinador que conoce las vistas y las notifica (ej. `AbstractTableModel`).
Swing (UI delegate)	**MVC puro**	En Swing, vista y controlador se fusionan en el delegado de interfaz de usuario.
`==`	`equals()`	`==` compara identidad (referencia); `equals` compara valor y, por defecto, equivale a `==`.
Clon superficial	**Clon profundo**	`Object.clone()` copia referencias (comparte objetos internos); el profundo clona además los atributos mutables (`buffer.clone()`).
`Cloneable`	`Comparable<T>`	`Cloneable` es marcadora (sin métodos; habilita `super.clone()`); `Comparable` tiene `compareTo(T)` y da el orden natural.
Serializable	**XMLEncoder/Decoder**	Serializable: binario, ligado a la estructura interna (frágil), solo Java. XML: guarda pasos vía API pública, legible, independiente de plataforma; exige ctor vacío + get/set.
`transient`	`static`	Ambos se **excluyen** de la serialización; `transient` vuelve al valor por defecto del tipo y `static` pertenece a la clase, no al objeto.
DAO	**DTO**	DAO: componente con comportamiento (CRUD) que accede a la fuente. DTO: objeto plano de datos, sin lógica, que viaja entre capas.
Creacional / Estructural / Comportamiento	(clasificación)	Creacional: cómo se crean objetos. Estructural: cómo se componen (ej. Decorator en los `Filter*Stream`). Comportamiento: cómo colaboran y se reparten responsabilidades (Observer, State).

Trampas Clásicas

TRAMPA CLÁSICA 1 **`wait()`/`notify()` solo dentro de código `synchronized`, y la condición siempre en un `while`.** Fuera de un bloque sincronizado lanzan `IllegalMonitorStateException`; con `if` en lugar de `while` el hilo asume una condición que otro pudo haber cambiado. Además, `start()` se llama **una sola vez** por hilo (si no, `IllegalThreadStateException`) y llamar `run()` directamente **no** crea un hilo; los constructores no pueden ser `synchronized`.

TRAMPA CLÁSICA 2 **`clone()`: empezar por `super.clone()`, nunca por `new`, y clonar a mano los atributos mutables.** `Object.clone()` es `protected` y superficial; sin `implements Cloneable` lanza `CloneNotSupportedException` (chequeada, hay que capturarla o declararla); una subclase no puede agregar excepciones chequeadas a un `clone()` redefinido; los arrays son objetos y se comparten si no se clonan.

TRAMPA CLÁSICA 3 **Redefinir `equals` obliga a redefinir `hashCode`, y `equals` recibe `Object`.** Si dos objetos son `equals` con distinto hash, `HashSet`/`HashMap` los tratan como distintos; con otro tipo de parámetro se está **sobrecargando**, no redefiniendo. Análogamente, en Observer olvidar `setChanged()` antes de `notifyObservers()` hace que **nadie sea notificado**, y en serialización un solo atributo no serializable provoca `NotSerializableException`.